

Design, Implementation and Evaluation of an Edge-AI Cluster Architecture for Performance-Optimized AI Workloads

Huy Tran An^{1,2}, Huy Tran Quang^{1,2}, Cuong Truong Van^{1,2}

¹ University of Information Technology, Ho Chi Minh City, Vietnam

² Vietnam National University, Ho Chi Minh City, Vietnam

22520574@gm.uit.edu.vn, 22520578@gm.uit.edu.vn, cuongtv@uit.edu.vn

Abstract—The surging demand for computer vision has created an urgent need for efficient processing of continuous high-definition video streams at the network edge. However, orchestrating computationally intensive AI inference tasks within distributed edge clusters constitutes a significant NP-hard scheduling problem, exacerbated by volatile resource states, strict energy constraints, and the substantial model-loading overhead (up to 400 ms per cold start) in multi-model workloads. This paper proposes a production-grade Edge-AI cluster architecture featuring a hybrid PSO-ACO-GA task scheduler specifically engineered for continuous task ingestion and asynchronous execution on real heterogeneous hardware. The system integrates a decentralized Gossip Protocol for fault-tolerant, near-real-time propagation of worker states (CPU load, residual energy, and model residency) to the manager node. Within this framework, Particle Swarm Optimization initializes high-quality pheromone trails, Ant Colony Optimization performs adaptive task assignment, and an asynchronous Genetic Algorithm dynamically tunes the governing parameters, while a composite heuristic incorporating batching bonuses and anti-bias penalties minimizes redundant model loading. Experimental evaluations on 2,000 continuous video analytics tasks demonstrate that the proposed hybrid approach consistently outperforms all baseline schedulers. It achieves the lowest makespan and the highest throughput, reducing makespan by up to 12.6% and improving throughput by up to 14.3% compared to conventional baselines, while consuming the lowest total energy among all policies. By unifying decentralized state sharing, model-affinity batching, and hybrid metaheuristic scheduling on real edge hardware, this work provides a practical and scalable solution for high-throughput, energy-aware distributed inference in next-generation Edge-AI systems.

Index Terms—Edge computing, Edge-AI, distributed inference, edge clusters, task scheduling

I. INTRODUCTION

The rapid proliferation of IoT devices and the explosive growth of high-definition video streams have created an urgent need for computer vision at the network edge. Centralized cloud processing introduces unacceptable latency and bandwidth overhead, prompting a shift toward distributed computing paradigms that bring processing closer to data sources. In particular, near-edge and in-network computing models have emerged as promising solutions, enabling computation on heterogeneous hardware platforms while significantly reducing data transmission costs [1]. Nevertheless, individual edge nodes often lack sufficient computational capacity to handle continuous high-definition video analytics. Techniques such as AI multi-tenancy, which allow multiple deep learning models to execute concurrently on a single device, have been shown to markedly improve inference throughput under severe resource constraints [2]. To scale beyond single-node limitations, edge devices are therefore organized into collaborative Edge-AI

clusters that perform distributed inference across multiple heterogeneous workers.

However, orchestrating continuous AI task streams within such clusters remains a formidable NP-hard scheduling problem. Workload volatility, dynamic resource availability, and strict energy budgets make traditional centralized schedulers ineffective. Moreover, the multi-stage nature of AI inference introduces substantial model-loading overhead (typically 200–500 ms per cold start), which becomes a dominant bottleneck in multi-model, continuous workloads. These challenges necessitate decentralized, adaptive scheduling mechanisms that can react to node states while exploiting model residency to minimize initialization penalties.

This paper presents the design, implementation, and evaluation of the hybrid PSO-ACO-GA scheduler on a real heterogeneous 17-node Milk-V Edge-AI cluster. The architecture uses a Gossip Protocol for fault-tolerant state propagation, with PSO initializing pheromone trails, ACO performing adaptive assignment, and asynchronous GA tuning parameters. Experiments on 2,000 tasks show superior makespan, throughput, and energy efficiency over baselines.

II. RELATED WORK

A. Edge AI Systems and On-Device Inference

Edge intelligence has become a cornerstone of modern AI deployment by moving inference closer to data sources, thereby reducing latency and bandwidth consumption [3]. Recent systematic reviews provide comprehensive taxonomies of Edge-AI architectures and deployment strategies. Gauttam et al. [4] analyze architectures, applications, and open challenges across the edge continuum, while Cordova-Cardenas et al. [5] focus on practical deployment frameworks for neural networks on embedded systems, emphasizing model optimization techniques such as pruning and quantization. Hao et al. [2] specifically investigate AI multi-tenancy on edge devices, demonstrating that concurrent execution of multiple models on a single node can substantially increase overall inference throughput. Despite these advances, existing works predominantly address single-node performance and lack mechanisms for continuous, heterogeneous task scheduling across distributed Edge-AI clusters.

B. Metaheuristic-Based Task Scheduling

Task allocation in edge environments has been extensively studied through systematic literature reviews and metaheuristic approaches. Patsias et al. [6] surveyed task allocation methods, highlighting that optimal strategies must balance

multiple conflicting objectives including energy efficiency, latency, QoS, and privacy depending on the underlying edge architecture. Recent hybrid metaheuristics have shown promise in addressing these trade-offs. Tripathy et al. [7] proposed EMAPSO, a bi-objective PSO variant that simultaneously minimizes makespan and energy on fog-enabled IoT nodes with heterogeneous processing capabilities. Sahu et al. [8] introduced a hybrid Cellular Potts Model and PSO framework that jointly optimizes energy and latency through spatial resource modeling. Similarly, Sanaj et al. [9] presented EcoTaskOpt, a hybrid ACO-PSO algorithm for cloud-edge task scheduling that employs a weighted multi-objective fitness function.

While these studies advance the state of the art, the majority rely on simulation environments and synthetic workloads. Energy models are frequently simplified, and the critical overhead of neural network model initialization in continuous video analytics workloads is rarely considered. Furthermore, few frameworks integrate decentralized coordination, model-affinity batching, and production-grade features within a single hybrid scheduler.

C. Gap Analysis and Contributions

Despite substantial progress in Edge-AI systems and metaheuristic scheduling, critical gaps remain. Most evaluations are conducted in simulated environments on homogeneous or mildly heterogeneous nodes, with limited validation on real heterogeneous hardware. Energy and latency models often overlook model-loading costs, and decentralized gossip-based state propagation is seldom combined with advanced hybrid optimization.

This paper addresses these gaps by implementing and evaluating the proposed hybrid metaheuristic scheduler (PSO-ACO-GA) on a real heterogeneous 17-node Milk-V Edge-AI cluster. The architecture integrates decentralized Gossip Protocol, PSO pheromone initialization, ACO adaptive assignment, asynchronous GA tuning, and a composite heuristic with batching bonuses.

III. SYSTEM MODEL AND PROBLEM FORMULATION

A. Architecture

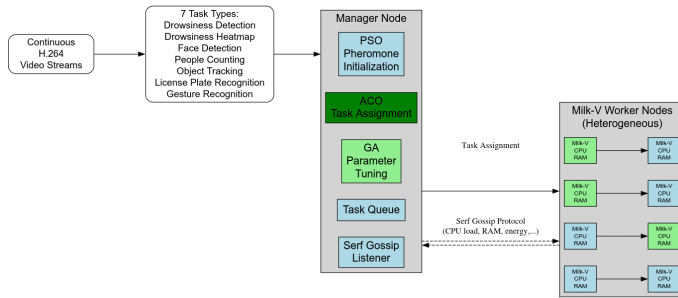


Fig. 1. Hybrid PSO-ACO-GA Edge-AI Cluster Architecture

The proposed system operates on a real heterogeneous 17-node Milk-V Edge-AI cluster for on-edge AI inference from H.264 video streams. As shown in Figure 1, a central manager node receives continuous video tasks and dispatches them to workers. The manager consists of five modules arranged vertically: Serf Gossip Listener, PSO Pheromone Initialization, ACO Task Assignment, GA Parameter Tuning, and Task Queue.

Node synchronization is realized through a decentralized gossip protocol [10]. Worker nodes periodically send state updates (CPU utilization, available RAM, power draw, and batching records) to the manager, providing high fault tolerance with minimal network overhead. The system assumes offline or buffered video streams, resident AI models on workers to avoid repeated loading, and dynamic resource monitoring. The primary goals are to minimize overall makespan and cluster-wide energy consumption rather than enforcing per-task deadlines.

B. Task Model

Inference workloads involve seven distinct video analytics tasks: *Drowsiness Detection*, *Heatmap Generation*, *Face Detection*, *People Counting*, *Object Tracking*, *License Plate Recognition*, and *Gesture Recognition*. Each workload is categorized into a specific *task type*.

Each task type corresponds to a deep neural network model pre-trained on the cloud and quantized to INT8 precision, making it suitable for efficient execution on resource-constrained Milk-V boards. Processing an individual task comprises three sequential phases: model initialization, execution (inference), and post-processing.

These workloads exhibit strong temporal locality; a sequence of related tasks typically follows an initial detection event. The cold-start initialization of a neural network introduces an average 0.4 s (400 ms) latency penalty, as measured on the target Milk-V nodes. The optimization strategy therefore focuses on maximizing the batching count of identical task types to exploit resident models and bypass redundant reloads. Workers can execute multiple tasks concurrently, provided sufficient RAM and CPU availability exist.

C. Parameters and Constraints

The key notations and parameters used throughout this study are summarized in Table I. Energy consumption is estimated using a calibrated EMAPSO-style analytical model [7]:

$$E = \sum_{i=1}^M \sum_{w=1}^N x_{i,w} \cdot \left(\alpha_e \cdot \hat{T}_{i,w} + \beta_e \cdot b_i \right) \quad (1)$$

where $\alpha_e = 1.0$ and $\beta_e = 5 \times 10^{-9}$. Communication energy remains negligible compared to computation and model-loading costs because all transmission occurs on a high-bandwidth private Docker bridge network. This analytical model is applied consistently across all scheduling policies in both real-hardware execution and simulation mode, ensuring fair and reproducible comparisons. The reported energy values are *estimated* rather than directly measured with hardware power sensors.

Hard constraints are enforced to prevent system overload. The assigned workload must not exceed the physical CPU or RAM capacities of worker w at any moment t :

$$\sum_{i \in \text{tasks assigned to } w} \text{cpu_req}_i \leq \text{CPU}_w^{\text{avail}}(t), \quad (2)$$

$$\sum \text{mem_req}_i \leq \text{Mem}_w^{\text{avail}}(t). \quad (3)$$

TABLE I
SUMMARY OF KEY NOTATIONS

Notation	Description
Cluster and Node Parameters	
N	Number of heterogeneous nodes (17 nodes)
$\text{CPU}_w^{\text{avail}}(t)$	Available CPU of worker w at time t (GHz)
$\text{Mem}_w^{\text{avail}}(t)$	Available memory of worker w at time t (MB)
ν_w	Speed of worker w (0.5–1.5)
Task Parameters	
k_t	Base processing ratio for task type t (7 types)
d_v	Video duration (seconds)
$\hat{T}_{t,w}$	Estimated execution time of type t on worker w
Energy Model (EMAPSO-style) [7]	
α_e	Computation cost coefficient (J per second)
β_e	Transmission cost coefficient
Algorithm Parameters	
α, β	ACO pheromone and heuristic factors
ρ	Pheromone evaporation rate
ω, c_1, c_2	PSO inertia weight and learning factors
τ_w	Pheromone intensity for worker w
η_w	Composite heuristic score for worker w
Evaluation Metrics	
L	Total makespan (seconds)
E	Total energy consumption
Gini	Gini index of task distribution

D. Problem Formulation

The dynamic assignment of streaming tasks to distributed worker nodes is formulated as a constrained multi-objective optimization problem. Let $\mathcal{T} = \{T_1, \dots, T_M\}$ represent the queue of incoming tasks, and let $\mathcal{W} = \{W_1, \dots, W_N\}$ denote the set of 17 available heterogeneous worker nodes. A binary decision variable $x_{i,w} \in \{0, 1\}$ signifies the assignment of task i to worker w .

The scheduling optimization objectives are the dual minimization of total makespan L and cluster-wide energy E (using the model in Eq. 1):

$$\min L = t_{\text{last}} - t_{\text{first}} \quad (4)$$

This formulation captures the inherent trade-offs required when scheduling tasks across a shared heterogeneous cluster. Task assignment in edge computing environments is known to be NP-hard [6]. Consequently, the proposed hybrid metaheuristic (PSO–ACO–GA) is employed to generate near-optimal schedules with decision latencies in the sub-second range, as validated in the experimental evaluation.

IV. HYBRID SCHEDULING ALGORITHM

A. Overview

The core of the proposed architecture is a hybrid PSO–ACO–GA scheduling engine executing on the central manager node. Its primary function is the intelligent, continuous distribution of streaming AI inference tasks across the heterogeneous 17-node Milk-V cluster. The logical workflow is outlined in Algorithm 1.

This hybrid design addresses the individual limitations of its constituent algorithms. Particle Swarm Optimization rapidly generates high-quality initial pheromone trails. Ant Colony Optimization performs adaptive, probabilistic task assignment that exploits both pheromone history and dynamic node states.

An asynchronous Genetic Algorithm continuously tunes the governing parameters α , β , and ρ . A composite heuristic score η_w integrates task-specific affinity, batching bonus, anti-bias penalty, load penalty, and energy-aware factors. This integrated structure limits routing latency to sub-second overhead per task while achieving broad adaptability across continuous Edge-AI workloads.

Algorithm 1: Hybrid PSO–ACO–GA Task Assignment

```

Function AssignTask(task):
    Retrieve cluster state via Serf gossip;
    affMap ← loadAffinities(task.type);
    projectedLoads ← calculateProjectedLoads();
    // Stage 1: PSO-based pheromone
    initialization
    initialPheromone ← psoPrior(scores, task,
    projectedLoads);
    // Stage 2: ACO task assignment
    bestWorker ← acoAssign(initialPheromone, task,
    projectedLoads);
    // Stage 3: Asynchronous GA
    parameter tuning
    if totalAssigned() mod 5 = 0 then
        | updateACOParamsWithGA();
    Update batching records and system metrics;
    updateBatchingRecords(bestWorker, task);
    return bestWorker;

```

B. Solution Encoding and Node Evaluation

In continuous edge video processing, each incoming task T_t is assigned dynamically upon arrival using live Serf gossip state (CPU load, free memory, residual energy, and recent task history). The core decision mechanism is a composite multi-faceted heuristic score η_w computed for every candidate worker $w \in \mathcal{W}$. This score integrates historical performance, model-affinity, batching opportunity, load balancing, and energy awareness, and is used both for ACO probabilistic selection and as part of the overall fitness landscape.

The general form of the composite heuristic is

$$\eta_w = \underbrace{\phi_{t,w}^\gamma}_{\text{affinity + pheromone}} \cdot \underbrace{(1 - L_w)^\delta}_{\text{load penalty}} \cdot M_w \cdot (1 + \kappa \cdot B_w) \cdot A_w \cdot S_{t,w} \cdot U_w / \left(\lambda + \frac{\hat{T}_{t,w}}{\mu} \right) / (1 + \nu \cdot E_w), \quad (5)$$

where the components are defined as follows (all symbols are summarized in Table I and the specific exponents/weights $\gamma, \delta, \kappa, \lambda, \mu, \nu$ are dynamically tuned by the asynchronous GA component):

- M_w : normalized available memory ratio $\frac{\text{freeMem}_w}{\text{memReq}_t}$,
- $\hat{T}_{t,w}$: predicted execution time of task type t on worker w ,
- E_w : energy proxy proportional to predicted execution time,
- L_w : projected normalized load after assignment,
- A_w : anti-bias penalty (Eq. 8),
- B_w : batching bonus rewarding temporal locality of identical tasks (Eq. 7),

- $S_{t,w}$: specialization term combining task complexity, pheromone intensity, and GA-derived fitness,
- U_w : usage-decay factor that discourages over-assignment to heavily loaded nodes.

1) *Affinity Component*: The affinity $\phi_{t,w}$ quantifies the worker's historical efficiency for task type t :

$$\phi_{t,w} = \frac{1/\overline{\text{sec}}_{t,w}}{\max_c(1/\overline{\text{sec}}_{t,c})} \times (1 + \theta(0.32 - u_w)), \quad (6)$$

where $\overline{\text{sec}}_{t,w}$ is the exponentially-weighted moving average execution time, u_w is the usage ratio, and θ is a tunable scaling factor.

2) *Batching Bonus*: The batching bonus exploits model residency:

$$B_w = \min \left(0.8, \frac{\text{count of recent identical tasks in window}}{4} \right). \quad (7)$$

3) *Anti-bias Penalty*: To promote load fairness:

$$A_w = \begin{cases} 1 + (0.3 - p_w) \times 1.5 & \text{if } p_w \leq 0.3, \\ \frac{1}{1 + (p_w - 0.3) \times 4} & \text{otherwise,} \end{cases} \quad (8)$$

with $p_w = \frac{\text{assigned tasks to } w}{\text{total assigned tasks}}$.

C. Hybrid Optimization Process

Upon task arrival, the hybrid PSO-ACO-GA engine executes three tightly integrated stages (Algorithm 1):

Stage 1 – PSO pheromone initialization: A lightweight particle swarm (augmented with GA-style crossover and mutation) rapidly constructs high-quality initial pheromone trails τ_w for the current task type.

Stage 2 – ACO probabilistic assignment: The best worker is selected using the classic ACO rule

$$\text{prob}_w = \frac{\tau_w^\alpha \cdot \eta_w^\beta}{\sum_{w'} \tau_{w'}^\alpha \cdot \eta_{w'}^\beta}, \quad (9)$$

followed by pheromone reinforcement that incorporates the batching bonus:

$$\tau_w^{(t+1)} = (1 - \rho) \tau_w^{(t)} + \Delta \tau_w, \quad (10)$$

where $\Delta \tau_w$ is proportional to the normalized heuristic score scaled by the batching opportunity.

Stage 3 – Asynchronous GA parameter tuning: Every five assigned tasks, an asynchronous genetic algorithm (population size 13, 18 generations) evolves the governing hyperparameters (α, β, ρ) . The fitness of each candidate triple is evaluated as

$$\text{Fitness}(\alpha, \beta, \rho) = \frac{1}{L} \times (1 - 1.5 \times \min(\text{load_variance}, 2/3)) \times (1 - 0.5 \times \text{bias_ratio}), \quad (11)$$

with a floor of 0.01 to ensure non-negative values suitable for tournament selection. Weighted arithmetic crossover and decaying Gaussian mutation maintain continuous adaptation without interrupting the main scheduling loop.

This three-stage design ensures sub-second decision latency while continuously balancing exploration (PSO), exploitation with model-affinity (ACO), and long-term hyperparameter adaptation (GA). The manager dispatches lightweight RPC metadata to the selected worker, and updated state is propagated back through the Serf gossip protocol, closing the feedback loop.

V. EXPERIMENTAL SETUP

A. Platform and Workload

The system was evaluated on a containerized Edge-AI cluster orchestrated by Docker Compose, consisting of one central edge-server and 17 heterogeneous Milk-V worker nodes (milkv-A to milkv-Q) connected through a private bridged network. Worker nodes were deliberately configured with different processing speeds and memory limits to reflect real-world heterogeneous edge hardware:

- `NODE_SPEED` ranged from 0.5 GHz (milkv-A) to 1.5 GHz (milkv-Q);
- `MEMORY_LIMIT` ranged from 128 MB (milkv-A) to 256 MB (milkv-Q).

All nodes continuously exchange state (CPU load, free memory, current task, batching count) via the Serf gossip protocol, ensuring scheduling decisions are based on near-real-time cluster state data.

The evaluated workload consisted of two workload sizes: 1,000, and 2,000 continuous AI inference tasks generated from **seven distinct video analytics task types**. The base execution durations (seconds) calibrated at unit node speed ($\nu_w = 1.0$) are:

$$k_t = \begin{cases} 1.82 & \text{Drowsiness Detection} \\ 3.59 & \text{Heatmap Generation} \\ 2.53 & \text{Face Detection} \\ 1.56 & \text{People Counting} \\ 1.34 & \text{Object Tracking} \\ 1.76 & \text{License Plate Recognition} \\ 3.20 & \text{Gesture Recognition.} \end{cases} \quad (12)$$

Tasks were generated using a fixed seed for reproducibility and the same task list was reused across all algorithms to ensure a fair comparison; independent runs varied the stochastic initialization seeds of PSO, ACO, and GA across three distinct seeds. Tasks were submitted in batches of 40 with a 20-second interval to simulate realistic continuous video stream ingestion.

B. Compared Algorithms

Six distinct algorithmic scheduling policies were evaluated under identical workload parameters: (i) PSO-ACO-GA (Proposed Method): The full hybrid framework combining PSO initialization, ACO adaptive assignment, and GA-based parameter tuning with batching and affinity mechanisms. (ii) PSO: A pure Particle Swarm Optimization approach where task assignments are determined by particle convergence toward heuristic optima. (iii) ACO: A standard Ant Colony Optimization policy without GA tuning or PSO initialization. (iv) GA: A standalone Genetic Algorithm where task-to-node mapping is optimized over generations. (v) Greedy: Upon task arrival, algorithmic complexity is bypassed, mapping the assignment deterministically to the node with the highest instantaneous heuristic score. (vi) EcoTaskOpt: A hybrid ACO-PSO algorithm employing a weighted multi-objective fitness function.

The system's performance was evaluated across five primary metrics: (i) Makespan (seconds): The total wall-clock time required to complete the entire batch of tasks. (ii) Latency (seconds): The mean per-task completion time from submission to result delivery, reflecting the responsiveness of the scheduler under continuous load. (iii) Gini Index: A statistical

TABLE II
PERFORMANCE COMPARISON OF SCHEDULING ALGORITHMS ON SYSTEM-LEVEL METRICS (1,000 TASKS SIMULATION, 17-WORKER EDGE-AI CLUSTER;
MEAN OF 10 INDEPENDENT RUNS OVER 10 RANDOM SEEDS).

Algorithm	Makespan (s) ↓	Latency (s) ↓	Total Energy (J) ↓	Gini Index ↓	Throughput ↑
ACO	600.42 ± 5.24	8.205 ± 0.091	7179.33 ± 84.21	0.137 ± 0.013	1.660 ± 0.015
ECOTASK	580.04 ± 5.43	8.155 ± 0.097	7140.90 ± 101.59	0.143 ± 0.014	1.711 ± 0.013
GA	604.47 ± 6.42	8.250 ± 0.064	7218.45 ± 60.88	0.133 ± 0.010	1.652 ± 0.018
GREEDY	617.89 ± 9.44	8.296 ± 0.066	7263.68 ± 62.24	0.126 ± 0.009	1.615 ± 0.023
PSO	672.49 ± 20.73	8.509 ± 0.061	7522.30 ± 73.95	0.135 ± 0.013	1.484 ± 0.046
PSO-ACO-GA	552.56 ± 8.57	8.056 ± 0.045	7045.86 ± 41.95	0.135 ± 0.007	1.807 ± 0.028

TABLE III
PERFORMANCE COMPARISON OF SCHEDULING ALGORITHMS ON SYSTEM-LEVEL METRICS (2,000 TASKS ON REAL HETEROGENEOUS 17-NODE MILK-V
EDGE-AI CLUSTER; MEAN OF 10 INDEPENDENT RUNS OVER 10 RANDOM SEEDS).

Algorithm	Makespan (s) ↓	Latency (s) ↓	Total Energy (J) ↓	Gini Index ↓	Throughput ↑
ACO	1150.43 ± 9.27	8.251 ± 0.054	14444.62 ± 107.31	0.127 ± 0.016	1.737 ± 0.015
ECOTASK	1130.39 ± 7.46	8.186 ± 0.059	14341.70 ± 133.87	0.124 ± 0.010	1.768 ± 0.010
GA	1156.78 ± 2.01	8.217 ± 0.082	14393.50 ± 139.80	0.134 ± 0.007	1.727 ± 0.004
GREEDY	1192.62 ± 4.67	8.334 ± 0.039	14559.51 ± 66.91	0.123 ± 0.003	1.675 ± 0.008
PSO	1218.78 ± 44.10	8.356 ± 0.056	14725.14 ± 185.17	0.134 ± 0.011	1.639 ± 0.059
PSO-ACO-GA	1065.74 ± 14.17	8.042 ± 0.069	14060.15 ± 123.02	0.138 ± 0.003	1.874 ± 0.023

measure of distribution inequality, indicating the equitability of task assignment across nodes. (iv) Total Energy Consumption (Joules): The estimated cumulative energy expenditure derived from task duration and node-level power models. (v) System Throughput (tasks/s): The rate at which the cluster successfully processes and resolves AI inference tasks. All reported metric values are averaged over ten independent runs across three distinct random seeds, and presented as mean ± standard deviation.

VI. RESULTS AND ANALYSIS

A. System-Level Performance

The continuous execution of 2,000 Edge-AI tasks on the real heterogeneous 17-node Milk-V cluster revealed clear performance advantages of our hybrid PSO-ACO-GA scheduler. As summarized in Table III, the proposed hybrid approach achieved the lowest makespan of 1065.74 s (±14.17). This represents a 5.7% improvement over ECOTASK (1130.39 s), a 7.4% improvement over ACO (1150.43 s), a 7.9% improvement over GA (1156.78 s), a 10.6% improvement over Greedy (1192.62 s), and a 12.6% improvement over pure PSO (1218.78 s).

The hybrid scheduler also delivered the highest throughput at 1.874 tasks/s (±0.023), outperforming Greedy by 11.9% (1.675 tasks/s) and pure PSO by 14.3% (1.639 tasks/s). Average latency was the lowest at 8.042 s (±0.069).

Total energy consumption is estimated using the EMAPSO-inspired analytical model, reaching 14060.15 J (±123.02) for PSO-ACO-GA—the smallest among all policies.

Gini index analysis shows PSO-ACO-GA maintained equitable load distribution (0.138 ± 0.003), competitive with Greedy (0.123 ± 0.003) while delivering significantly better makespan and throughput.

B. Ablation Study

To quantify the individual contribution of each module in the hybrid PSO-ACO-GA scheduler, we conducted a controlled ablation study on 1,000 continuous video analytics tasks (Table IV). We evaluated three configurations: the full

hybrid (PSO-ACO-GA), the variant without asynchronous GA parameter tuning (PSO-ACO-GA_NoGA), and the variant without PSO-based pheromone initialization (PSO-ACO-GA_NoPSO).

The full hybrid achieves the best overall performance, with the highest throughput (1.835 jobs/s) and lowest makespan (545.8 s). Disabling GA parameter tuning (NoGA) causes the most significant degradation—throughput drops to 1.771 jobs/s (3.5% lower) and makespan rises to 565.5 s (3.6% higher)—while removing PSO initialization (NoPSO) yields milder degradation (1.783 jobs/s and 561.7 s, 2.8% and 2.9% worse, respectively).

For load balancing, the full hybrid achieves the best Gini index of 0.132, while NoGA shows the worst distribution (0.158), underscoring the critical role of GA-driven parameter tuning. NoPSO yields an intermediate Gini of 0.142, confirming that PSO initialization contributes to, but is not solely responsible for balanced assignment. These results confirm the synergistic effect of the three metaheuristics: GA tuning is the dominant factor for throughput, makespan, and load fairness, while PSO initialization provides complementary efficiency gains.

C. Scalability and Consistency Analysis

To assess the scalability of the proposed scheduler, we evaluated system performance under two workloads of increasing size (1,000 and 2,000 tasks) while keeping all other experimental conditions identical. Figs. 2–3 and Tables II–III present the detailed comparison.

As workload doubled from 1,000 to 2,000 tasks, the proposed PSO-ACO-GA scheduler maintained its leading position across all key metrics. Makespan increased only 92.9% (from 552.56 s to 1065.74 s), demonstrating clearly sub-linear scaling. Throughput improved from 1.807 to 1.874 tasks/s, confirming that the hybrid engine becomes more efficient as load grows. Energy consumption scaled proportionally (from 7045.86 J to 14060.15 J), yet remained the lowest among all policies in both cases.

TABLE IV
ABLATION STUDY RESULTS (1,000 TASKS; MEAN OF 10 INDEPENDENT RUNS OVER 10 RANDOM SEEDS).

Scheduler	Throughput (jobs/s) $\uparrow$	Makespan (s) $\downarrow$	Gini Index $\downarrow$
PSO-ACO-GA (full)	1.835	545.8	0.132
PSO-ACO-GA_NoGA	1.771	565.5	0.158
PSO-ACO-GA_NoPSO	1.783	561.7	0.142

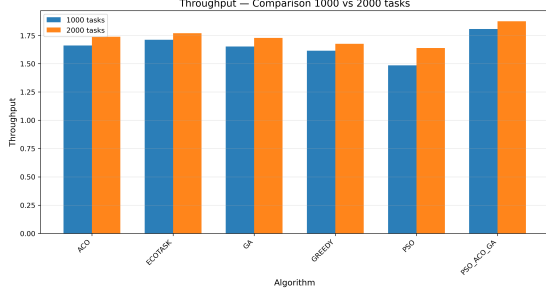


Fig. 2. Throughput comparison between 1,000-task and 2,000-task workloads.

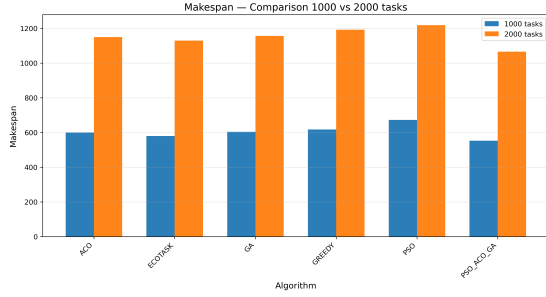


Fig. 3. Makespan comparison between 1,000-task and 2,000-task workloads.

D. Discussion

The proposed PSO-ACO-GA scheduler provides the best trade-off among makespan, throughput, energy, and load balancing by combining decentralized state sharing with model-affinity batching. Although its Gini index is not always the lowest, the design intentionally prioritizes makespan and throughput over perfect load uniformity, which is better suited for continuous video analytics workloads.

The single manager may become a bottleneck in larger deployments, and the evaluation assumes a high-bandwidth local network without accounting for bandwidth fluctuations, thermal conditions, or hardware health metrics.

VII. CONCLUSION AND FUTURE WORK

This paper presented a hybrid PSO-ACO-GA scheduler for Edge-AI clusters. Experimental evaluation on a real heterogeneous 17-node Milk-V cluster showed up to 12.6% reduction in makespan and 14.3% improvement in throughput compared with baseline schedulers, while also achieving the lowest energy consumption.

Future work will extend the architecture toward multi-tier edge-cloud deployment and incorporate dynamic metrics such as bandwidth and thermal states into the scheduling process to further improve scalability, responsiveness, and long-term system reliability.

VIII. ACKNOWLEDGEMENT

This research is funded by University of Information Technology-Vietnam National University Ho Chi Minh City under grant number D1-2026-15.

REFERENCES

- [1] R. A. Cooke and S. A. Fahmy, "A model for distributed in-network and near-edge computing with heterogeneous hardware," *Future Generation Computer Systems*, vol. 105, pp. 395–409, 2020.
- [2] J. Hao, P. Subedi, L. Ramaswamy, and I. K. Kim, "Reaching for the sky: Maximizing deep learning inference throughput on edge devices with ai multi-tenancy," *ACM Transactions on Internet Technology*, vol. 23, pp. 2:1–2:33, Feb. 2023.
- [3] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, pp. 1738–1762, Aug. 2019.
- [4] H. Gauttam, G. Nain, K. K. Pattanaik, and P. Mendes, "Edge-ai: A systematic review on architectures, applications, and challenges," *Journal of Network and Computer Applications*, vol. 245, p. 104375, 2026.
- [5] R. Cordova-Cardenas, D. Amor, and Gutiérrez, "Edge ai in practice: A survey and deployment framework for neural networks on embedded systems," *Electronics*, vol. 14, no. 24, p. 4877, 2025.
- [6] V. Patsias, P. Amanatidis, D. Karampatzakis, T. Lagkas, K. Michalakopoulou, and A. Nikitas, "Task allocation methods and optimization techniques in edge computing: A systematic review of the literature," *Future Internet*, vol. 15, no. 8, p. 254, 2023.
- [7] N. Tripathy, S. Sahoo, N. S. Alghamdi, W. Viriyasitavat, and G. Dhiman, "Energy and makespan optimised task mapping in fog enabled iot application: a hybrid approach," *Scientific Reports*, 2026.
- [8] D. Sahu, Nidhi, S. Prakash, P. Sinha, T. Yang, R. S. Rathore, and L. Wang, "Beyond boundaries: a hybrid cellular potts and particle swarm optimization model for energy and latency optimization in edge computing," *Scientific Reports*, vol. 15, no. 1, p. 6266, 2025.
- [9] M. S. Sanaj *et al.*, "Energy-efficient task scheduling in cloud computing with ecotaskopt: A hybrid ant colony and particle swarm optimization approach," *International Journal of Computational Intelligence Systems*, 2026.
- [10] M. Franceschelli, A. Giua, and C. Seatzu, "Load balancing on networks with gossip-based distributed algorithms," in *Proceedings of the 46th IEEE Conference on Decision and Control*, pp. 500–505, IEEE, Dec. 2007.
- [11] A. Tundo, F. Filippini, F. Regonesi, M. Ciavotta, and M. Savi, "Decentralized edge workload forecasting with gossip learning," *IEEE Transactions on Network and Service Management*, 2025.
- [12] H. Izakian, A. Abraham, and V. Snášel, "Metaheuristic based scheduling meta-tasks in distributed heterogeneous computing systems," *Sensors*, vol. 9, no. 7, pp. 5339–5350, 2009.
- [13] H. Chen, A. M. K. Cheng, and Y.-W. Kuo, "Assigning real-time tasks to heterogeneous processors by applying ant colony optimization," *Journal of Parallel and Distributed Computing*, vol. 71, no. 1, pp. 132–142, 2011.